

• APSIS

Autonomous Space Traffic Management.
Collision-free orbits, computed in real time.

PROBLEM

Orbit is filling with bullets.

30,000+

tracked objects in orbit

14 km/s

typical closing speed in LEO

3

events made most of today's debris:
2007, 2009, 2021

Manual

operators still clear conjunctions by hand

Mega-constellations are adding tens of thousands of satellites. Human-in-the-loop avoidance does not scale. One uncontrolled collision can trigger a chain reaction that closes an orbital shell for decades, **Kessler syndrome**.

INSIGHT

Space needs air-traffic control ,
and it has to be **autonomous**.

Predict every close approach. Quantify the real collision probability.
Decide and execute the cheapest maneuver that removes the risk,
without waiting for a human.

SOLUTION

The full operational loop, running on real data.

1 • Ingest & Propagate

Live NORAD catalog, 8,400+ real objects, SGP4/SDP4 on every one.

2 • Screen

Protected-asset-vs-catalog and all-pairs spatial-hash sieve across a shell.

3 • Assess

TCA, miss distance, and Foster 2D collision probability, the operational standard.

4 • Decide

Minimum-propellant avoidance maneuver, optimized and explained.

5 • Act

Post-burn trajectory recomputed and verified below the safe line.

6 • Visualize

Real-time 3D mission-control globe of the whole picture.

REAL RESULT · REPRODUCIBLE

An active satellite, on a collision course with ASAT debris.

| QIANFAN-168 × FENGYUN 1C debris

0.31 km

miss distance

13.1 km/s

relative speed

1.07×10^{-4}

collision probability, above the action line

< 1 m/s

burn APSIS uses to clear it

Found by re-running `scripts/find-events.ts` on the bundled catalog.

Nothing here is staged.

Foster 2D collision probability.

At closest approach the relative motion is linear. We collapse the 3D geometry onto the **encounter plane** perpendicular to the relative velocity, project the combined position covariance into that plane, and integrate the 2D Gaussian over the combined hard-body disk.

- › Direct polar quadrature, exact in the fine limit
- › The same method NASA CARA and ESA run operationally
- › Covariance model documented and shown in the UI, not hidden

```
// encounter-plane integral
Pc = ∫∫|x|≤R N(x; μ, C2) dx

// cascade of filters
perigee/apogee sieve →
adaptive TCA march →
golden-section refine →
Foster Pc
```

Minimum-propellant maneuver, anchored to truth.

- › Searches lead time $\times$ in-track Δv , the minimum-energy axis for separation
- › Bisection for the smallest burn that drives P_c below the safe target
- › Propagates the **actual post-burn orbit** with a universal-variable two-body solver
- › **Differential correction** against SGP4 cancels model error to first order
- › Reports Δv , lead time, propellant, before/after P_c , and explains itself

"...a 4.2 cm/s prograde in-track burn executed 95 min before closest approach widens the miss to 8.6 km and lowers probability to 9.1×10^{-7} , a 1.2×10^2 reduction, at a propellant cost of 9.7 g for a 500 kg / 220 s-Isp bus."

, generated decision rationale, citing exact optimizer outputs

VALIDATION

The engine is tested, not asserted.

8 / 8 passing

astrodynamics validation suite (Vitest)

< 5 cm

two-body orbit closure over one full period

1 part in 10^6

energy & angular-momentum conservation

reproducible

every featured conjunction re-derivable from a script

Strict TypeScript, no any in the engine, pure functions with no UI dependencies, the physics is a library you could lift out and reuse.

DEMO

Live mission control.

3D operations view

8,000 objects as a GPU point cloud, real orbits, atmosphere shader, 60 fps.

Conjunction watchlist

Ranked by collision probability, color-coded by severity, live countdown to TCA.

Autonomous response

One click plans the burn; the cyan arc is the recomputed post-burn trajectory.

► Switch to the live app for the walkthrough.

TECH STACK

Built to ship and scale.

Engine

TypeScript, SGP4 (satellite.js), custom conjunction / Pc / maneuver code

Compute

Web Worker, transferable buffers, ~12 Hz full-catalog propagation

Render

Three.js, custom point & atmosphere shaders

App

Next.js 15, React 19, Zustand, Tailwind; deploys to the edge

No backend needed for the demo. The whole pipeline runs client-side, which is also the proof that it is fast.

ARCHITECTURE

Clean separation: physics, compute, view.

```
React UI —state→ Zustand —msg→ Engine Web Worker
|                                     |
|←— position buffers / results —|
|
lib/ pure tested astrodynamics (no UI deps)
astro · conjunction · maneuver · math
```

The **lib/** core has zero UI dependencies and full test coverage, the same code runs in the app, the worker, the screening scripts, and the test suite.

APSYS Ground Node, closing the loop.

A low-cost, GPS-disciplined 137/435 MHz receiver that observes real satellite passes and feeds Doppler measurements back to APSIS.

- › Measured Doppler refines orbit tracks
- › Better tracks shrink the covariance the screening uses
- › A mesh of nodes becomes a contributing sensor network

2.3 dB

system noise figure (cascade design)

16 dB

CNR margin @ 1000 km beacon

4-layer

controlled-impedance RF board

BOM

real parts, schematic, netlist included

ROADMAP

From MVP to orbital infrastructure.

Round 1, now

Working platform: screening, probability, autonomous maneuver, validated engine, 3D ops view.

Round 2

CDM/operator covariance ingest, multi-asset fleet protection, maneuver scheduling, ground-node PCB layout + gerbers.

Round 3

Continuous OD from a ground-node mesh, constellation-scale autonomy, maneuver de-confliction across thousands of assets.



Real data. Real physics. Real autonomy.
Keeping orbit usable for the next generation.

Space & Aerospace • FAR AWAY 2026